



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра алгоритмических языков

Григорьев Денис Андреевич

**Методы обзорного реферирования
для построения энциклопедических статей Wikipedia**

КУРСОВАЯ РАБОТА

Научный руководитель:
к.ф.н. Добров Борис Викторович

Москва, 2026

Аннотация

Методы обзорного реферирования
для построения энциклопедических статей Wikipedia

Григорьев Денис Андреевич

В работе рассматривается задача многошагового поиска обзорного материала по заданному запросу. Практическая часть работы направлена на подготовку основы для дальнейших экспериментов: загрузку части русскоязычной коллекции FineWeb2, LLM-разметку качества сайтов, обучение модели RuModernBERT и начальную работу с тематическими тегами.

Содержание

1	Постановка задачи	5
2	Реферирование длинных текстов	7
2.1	Набор данных RuBookSum	7
2.2	Методы реферирования	8
2.3	Метрики качества	9
2.4	Параметры экспериментов	9
2.5	Результаты экспериментов	10
3	Методы поиска источников для энциклопедических статей Wikipedia	13
3.1	Обзор существующих подходов	14
3.2	Предварительные эксперименты с многошаговым поиском	16
3.3	Демонстрационный запуск TreeHop	16
3.4	Качественный анализ новостного примера	16
3.5	Подготовка русскоязычной поисковой коллекции	18
3.6	Автоматическая оценка качества источников	19
3.7	Статистика автоматической оценки качества сайтов	20
3.8	Применение модели BERT для классификации сайтов	20
3.9	Использование тематических тегов сайтов	22
4	Заключение	25
	Список литературы	26

Введение

В условиях быстрого роста объёмов текстовой информации задача автоматического построения обзорных материалов становится всё более значимой. Пользователь, сталкивающийся с новой темой, новостной статьёй, отдельным фактом или краткой энциклопедической заметкой, часто заинтересован не только в получении короткого ответа, но и в построении связного текста, раскрывающего тему с разных сторон. Такой текст должен включать основные факты, исторический и тематический контекст, связи с другими объектами и выводы, которые можно сделать на основе нескольких источников.

Одним из естественных применений такой постановки является построение энциклопедических статей в стиле Wikipedia. Энциклопедическая статья отличается от обычного краткого пересказа тем, что она должна быть структурированной, нейтральной и опираться на несколько источников. В ней требуется не только сжать уже имеющийся текст, но и собрать материалы по связанным аспектам темы, выделить существенную информацию и представить её в виде цельного обзорного изложения.

В рамках данной работы рассматриваются два взаимосвязанных направления. Первое направление связано с обзорным реферированием длинных текстов. Для этого исследуются методы, позволяющие обрабатывать документы, длина которых превышает контекстное окно современных языковых моделей. В качестве экспериментальной основы рассматривается задача реферирования длинных художественных произведений на русском языке и корпус RuBookSum. Хотя художественные тексты отличаются от энциклопедических источников по жанру, они обладают важным для данной работы свойством: ключевая информация распределена по большому объёму текста и не может быть надёжно извлечена одним прямым обращением к модели.

Второе направление связано с поиском источников для построения обзорного материала. Для энциклопедической статьи часто недостаточно одного документа или одного поискового запроса. Необходимо найти материалы, раскрывающие разные аспекты темы, а затем объединить их в итоговый текст. Поэтому в работе отдельно рассматриваются методы многошагового поиска, при котором система последовательно формирует связанные подзапросы и использует найденные документы для уточнения дальнейшего направления поиска.

Цель данной работы состоит в исследовании методов обзорного реферирования и поиска источников, применимых к построению энциклопедических статей Wikipedia.

1. Постановка задачи

В данной работе рассматривается задача построения обзорного реферата по заданной теме с возможностью дальнейшего использования такого реферата как основы для энциклопедической статьи Wikipedia. В качестве входа может выступать тема статьи, новостной факт, краткое описание объекта или уже существующий фрагмент энциклопедического текста. Целью является не получение короткого ответа на вопрос, а построение связного и структурированного материала, раскрывающего исходную тему через набор связанных подтем и источников.

Пусть задана текстовая коллекция

$$\mathcal{D} = \{d_1, d_2, \dots, d_N\},$$

где каждый документ d_i представляет собой текст или фрагмент текста из поисковой коллекции. Входную тему обозначим как q_0 . Для построения энциклопедического материала необходимо получить набор релевантных источников, определить структуру будущего текста и сгенерировать итоговый обзорный реферат.

В общем виде задача может быть представлена как построение последовательности поисковых запросов

$$Q = (q_0, q_1, \dots, q_T),$$

где каждый следующий запрос q_t уточняет, расширяет или тематически развивает исходную тему q_0 с учётом информации, найденной на предыдущих шагах. Для каждого запроса выполняется поиск документов:

$$\mathcal{R}_t = \text{Retrieve}(q_t, \mathcal{D}, k),$$

где k — число извлекаемых документов или фрагментов на одном шаге поиска. Совокупность найденных материалов обозначим как

$$\mathcal{R}(Q) = \bigcup_{t=0}^T \mathcal{R}_t.$$

Итоговым результатом является обзорный реферат

$$S = \text{Summarize}(q_0, \mathcal{R}(Q)),$$

который должен удовлетворять нескольким требованиям. Во-первых, он должен быть информативным и покрывать основные аспекты темы. Во-вторых, он должен быть связным и структурированным, то есть не сводиться к набору разрозненных фактов. В-третьих, он должен сохранять

нейтральный стиль, характерный для энциклопедического текста. В-четвёртых, он должен опираться на найденные источники и минимизировать появление неподтверждённых утверждений.

Таким образом, задача обзорного реферирования для построения энциклопедической статьи включает две подзадачи. Первая подзадача состоит в генерации качественного реферата по одному или нескольким длинным документам. Вторая подзадача состоит в поиске и отборе источников, необходимых для раскрытия разных аспектов темы. В последующих разделах сначала рассматриваются методы реферирования длинных текстов, а затем методы поиска источников.

2. Реферирование длинных текстов

Одной из основных трудностей при построении энциклопедических статей является работа с длинными источниками. Материалы часто превышают контекстное окно языковой модели или содержат большое количество второстепенной информации. Поэтому прямое применение модели к полному документу не всегда возможно и требует специальных стратегий разбиения, промежуточного сжатия и объединения результатов.

Для исследования этой проблемы в работе рассматривается задача реферирования длинных русскоязычных текстов. В качестве экспериментальной основы используется корпус RuBookSum, предназначенный для автоматического реферирования художественной литературы.

2.1. Набор данных RuBookSum

На момент начала исследования отсутствовал открытый и репрезентативный корпус, специально предназначенный для задачи реферирования художественных текстов на русском языке. Для проведения экспериментов был собран собственный корпус RuBookSum, состоящий из пар “текст книги — краткий пересказ”.

В качестве источника эталонных рефератов использовался ресурс “Народный Брифли”, где пользователи публикуют краткие пересказы литературных произведений. Такие пересказы являются естественными примерами абстрактивного реферирования: они не всегда дословно повторяют исходный текст, но передают основные события, факты и выводы. При этом их стиль и объём могут различаться: часть пересказов охватывает произведение целиком, а часть делит содержание на отдельные главы.

Исходные художественные тексты были получены из электронной библиотеки LibRuSec. Отбор произведений выполнялся на основе наличия соответствующего пересказа. Для более точного сопоставления книг и рефератов использовалось семантическое сходство: имена авторов из разных источников переводились в эмбединги с помощью SentenceTransformer, после чего сравнивались по косинусной мере. Тексты дополнительно очищались от метаинформации, HTML-тегов, служебных пометок и технических вставок.

В результате был получен корпус, включающий более 600 очищенных пользовательских пересказов, исходные произведения из LibRuSec и более 40 жанров. Общая статистика корпуса приведена в таблице 1.

Датасет	Число документов	Средняя длина документа	Средняя длина реферата
RuBookSum	634	35052.64	700.77
BookSum	405	112885.15	1167.20
Gazeta	60964	632.77	41.94

Таблица 1: Сравнение RuBookSum с другими датасетами для реферирования

2.2. Методы реферирования

В работе рассматривались четыре стратегии реферирования длинных текстов: базовый иерархический метод, иерархический метод с фильтрацией узлов, чертёжный метод Text-Blueprint и чертёжный метод с кластеризацией вопросов.

Базовый иерархический метод строится на последовательном сжатии текста. Сначала исходный документ разбивается на фрагменты фиксированной длины. Для каждого фрагмента генерируется локальный реферат. Затем локальные рефераты объединяются в группы, после чего модель снова строит реферат уже по промежуточным результатам. Процесс повторяется до тех пор, пока не будет получен один итоговый реферат всего произведения. Такой подход позволяет работать с документами, длина которых превышает контекстное окно модели.

Недостаток базового иерархического метода состоит в том, что на промежуточных уровнях могут сохраняться избыточные или близкие по смыслу фрагменты. В художественных текстах это особенно заметно, так как отдельные эпизоды могут повторять уже известные сведения или не вносить существенного вклада в развитие сюжета. Для решения этой проблемы был рассмотрен иерархический метод с фильтрацией узлов. В нём на каждом уровне иерархии вычисляются эмбединги промежуточных рефератов, после чего близкие по косинусной мере элементы отбрасываются как избыточные. Это позволяет уменьшить объём промежуточного представления и снизить время генерации.

Чертёжный метод Text-Blueprint использует промежуточный план перед генерацией итогового текста. Такой план строится в виде набора вопросно-ответных пар, отражающих ключевые события, темы и сущности исходного текста. Затем реферат генерируется не только по исходному фрагменту, но и по этому плану. Преимущество такого подхода состоит в большей управляемости генерации: модель получает явную структуру, которую необходимо раскрыть в итоговом тексте.

Модифицированный чертёжный метод с кластеризацией вопросов направлен на сниже-

ние избыточности плана. В базовой версии Text-Blueprint вопросы генерируются для каждого фрагмента отдельно, поэтому они могут повторяться или частично противоречить друг другу. Для устранения этой проблемы вопросы кодируются в эмбединги и кластеризуются алгоритмом KMeans. После этого для каждого кластера формируется обобщённый вопрос, а итоговый план строится уже на основе таких обобщённых вопросов. Это уменьшает число обращений к языковой модели и повышает согласованность структуры реферата.

2.3. Метрики качества

Для сравнения методов использовались несколько групп метрик, отражающих разные свойства итогового реферата. Поверхностное сходство с эталонным рефератом оценивалось с помощью ROUGE-L, основанной на длине наибольшей общей подпоследовательности между сгенерированным и эталонным текстом. Семантическая близость оценивалась с помощью BERTScore, где сравнение выполняется по близости эмбедингов токенов.

Кроме стандартных метрик использовались две метрики, ориентированные на проверку содержательного покрытия. Первая из них, Coverage, измеряет долю ключевых вопросов, на которые можно найти ответ в сгенерированном реферате:

$$\text{Coverage} = \frac{\#\{q_i : P(\text{“да”} \mid q_i, S) > 0.75\}}{N},$$

где N — общее число заранее сгенерированных вопросов, а $P(\text{“да”} \mid q_i, S)$ — вероятность того, что ответ на вопрос q_i содержится в тексте S .

Вторая метрика, AnswerSimilarity, измеряет среднее семантическое сходство между ответами, извлечёнными из сгенерированного реферата, и эталонными ответами:

$$\text{AnswerSimilarity} = \frac{1}{N} \sum_{i=1}^N \text{sim}(a_i^{\text{pred}}, a_i^{\text{ref}}),$$

где a_i^{pred} — ответ, полученный по сгенерированному реферату, a_i^{ref} — эталонный ответ, а sim — косинусное сходство эмбедингов.

2.4. Параметры экспериментов

Эксперименты проводились на тестовой части корпуса RuBookSum. Для ограничения вычислительных затрат рассматривались тексты длиной не более 800000 символов, а длина генерируемого реферата ограничивалась 500 словами. Исходный текст разбивался на фрагменты

по 2000 токенов. Токенизация выполнялась с помощью AutoTokenizer модели DeepPavlov/rubert-base-cased. Для воспроизводимости использовался фиксированный seed, равный 42.

В иерархическом методе с фильтрацией узлов порог косинусного сходства был установлен равным $\theta = 0.85$. Если для промежуточного реферата существовал предыдущий реферат с косинусным сходством выше этого порога, такой элемент считался избыточным и не использовался на следующем уровне объединения. В чертёжном методе с кластеризацией число кластеров выбиралось по эвристике

$$n_{clusters} = \max(2, \lceil \sqrt{N_{questions}} \rceil),$$

где $N_{questions}$ — число вопросов, сгенерированных по всем фрагментам до кластеризации.

В экспериментах использовались модели RuadaptQwen2.5-7B-Lite-Beta, RuadaptQwen3-32BInstruct-v2, DeepSeek V3, Qwen3-235B-A22B, tpro и yagpt5lite. Для каждой комбинации модели и метода измерялись BERTScore, ROUGE-L, Coverage, AnswerSimilarity и среднее время генерации.

2.5. Результаты экспериментов

Полученные результаты показывают, что разные метрики выделяют разные свойства моделей и методов. По метрикам фактологического покрытия наиболее сильный результат показала модель Qwen3-235B-A22B в чертёжном методе: для неё Coverage составила 54.78, а AnswerSimilarity — 43.99. Это означает, что построение промежуточного вопросно-ответного плана помогает крупной модели лучше сохранять ключевую информацию при генерации итогового реферата.

При этом по соотношению качества и времени генерации наиболее практичным оказался иерархический метод с фильтрацией узлов. Он уменьшает количество избыточных промежуточных фрагментов и тем самым снижает вычислительные затраты. Например, для Qwen3-235B-A22B время генерации в иерархическом методе с фильтрацией составило 83.06 секунды против 103.49 секунды в базовом иерархическом методе, при сохранении близких значений BERTScore и ROUGE-L. Для DeepSeek V3 аналогично наблюдалось снижение среднего времени с 196.77 до 147.21 секунды.

Интересно, что высокие значения ROUGE-L и BERTScore не всегда совпадают с лучшим покрытием ключевых вопросов. Например, yagpt5lite показала высокие значения поверхностных метрик, однако уступила более крупным моделям по Coverage и AnswerSimilarity. Это под-

тверждает необходимость использовать специальные метрики содержательного покрытия, а не ограничиваться только сравнением текстового сходства.

Отдельные замеры времени на контрольном тексте также подтвердили пользу модификаций. В частности, для текста размером 81049 символов и 11 фрагментов иерархический метод с фильтрацией оказался быстрее базового иерархического метода для всех рассмотренных моделей. Для DeepSeek V3 время снизилось с 237.83 до 72.42 секунды, для Qwen3-235B-A22B — со 113.24 до 39.45 секунды, а для yaqpt5lite — с 34.17 до 14.08 секунды.

Таким образом, эксперименты на RuBookSum показывают, что методы реферирования длинных текстов можно условно разделить по назначению. Если требуется максимальное содержательное качество и доступны ресурсы для использования крупной модели, целесообразен чертёжный подход. Если же важен баланс между качеством и скоростью, более подходящим оказывается иерархический метод с фильтрацией узлов. Для задачи построения энциклопедических статей это различие важно: на этапе предварительной обработки большого числа источников предпочтительны быстрые методы, а на этапе финальной генерации статьи может быть оправдан более дорогой метод с явным планом.

Таблица 2: Результаты по методам и моделям

Модель	Метрики	Иерархический	Чертежный	Иерархический с фильтрацией	Чертежный с кластеризацией
DeepSeek V3	bertscore	<u>60.0 ± 3.1</u>	58.0 ± 4.0	60.0 ± 2.9	58.4 ± 3.6
	rouge-l	13.7 ± 3.9	12.6 ± 4.6	13.5 ± 3.7	11.2 ± 3.9
	coverage	53.57 ± 21.66	40.19 ± 23.68	45.00 ± 23.03	34.68 ± 23.77
	similarity	<u>42.38 ± 17.73</u>	32.31 ± 19.33	35.64 ± 18.88	<u>27.76 ± 19.75</u>
	time	196.77 ± 187.85	315.67 ± 321.89	147.21 ± 146.4	132.60 ± 197.25
Qwen3-235B-A22B	bertscore	61.2 ± 3.0	<u>61.6 ± 3.3</u>	<u>60.9 ± 2.7</u>	<u>59.3 ± 3.4</u>
	rouge-l	<u>14.9 ± 4.0</u>	<u>15.8 ± 4.5</u>	<u>14.8 ± 3.7</u>	<u>12.2 ± 3.6</u>
	coverage	52.48 ± 20.79	54.78 ± 21.16	44.54 ± 23.03	30.19 ± 21.96
	similarity	41.68 ± 17.18	<u>43.99 ± 17.54</u>	<u>35.67 ± 18.87</u>	24.10 ± 17.62
	time	103.49 ± 97.30	230.35 ± 271.03	83.06 ± 102.05	158.30 ± 196.35
RuadaptQwen3-32B Instruct-v2	bertscore	57.3 ± 2.9	58.9 ± 3.6	57.7 ± 3.3	55.3 ± 3.3
	rouge-l	11.0 ± 2.4	10.6 ± 3.2	10.7 ± 2.4	7.8 ± 2.1
	coverage	33.12 ± 21.50	33.18 ± 22.83	32.19 ± 22.52	17.72 ± 15.23
	similarity	25.25 ± 16.94	26.21 ± 18.22	24.82 ± 17.74	13.97 ± 12.39
	time	218.30 ± 195.16	379.24 ± 500.40	166.79 ± 164.61	286.35 ± 395.97
tpro	bertscore	<u>59.4 ± 3.0</u>	<u>59.0 ± 4.9</u>	<u>59.5 ± 3.3</u>	<u>58.2 ± 3.7</u>
	rouge-l	<u>13.8 ± 3.1</u>	<u>14.7 ± 4.9</u>	<u>13.5 ± 3.0</u>	<u>11.8 ± 3.9</u>
	coverage	<u>40.27 ± 20.23</u>	<u>40.83 ± 22.42</u>	<u>37.13 ± 20.72</u>	<u>26.03 ± 18.44</u>
	similarity	<u>31.77 ± 16.63</u>	<u>32.60 ± 18.57</u>	<u>29.44 ± 16.83</u>	<u>20.83 ± 15.26</u>
	time	367.32 ± 324.49	592.39 ± 772.19	267.73 ± 253.34	247.59 ± 361.20
RuadaptQwen2.5-7B Lite-Beta	bertscore	55.4 ± 2.9	56.1 ± 4.9	55.8 ± 2.9	54.0 ± 4.0
	rouge-l	8.6 ± 2.5	10.1 ± 3.9	8.7 ± 2.5	7.7 ± 2.8
	coverage	19.66 ± 17.77	24.94 ± 21.08	20.31 ± 17.95	15.51 ± 14.83
	similarity	15.16 ± 14.11	20.03 ± 17.50	15.94 ± 14.39	12.23 ± 12.30
	time	68.86 ± 64.85	126.84 ± 145.74	53.59 ± 47.28	76.66 ± 91.78
yagpt5lite	bertscore	<u>62.5 ± 3.5</u>	<u>61.1 ± 3.8</u>	<u>62.1 ± 3.2</u>	<u>61.5 ± 3.3</u>
	rouge-l	<u>16.9 ± 5.1</u>	<u>15.8 ± 5.1</u>	<u>16.4 ± 4.7</u>	<u>14.3 ± 4.4</u>
	coverage	<u>36.85 ± 19.40</u>	<u>33.17 ± 21.58</u>	<u>31.75 ± 20.06</u>	<u>24.28 ± 16.95</u>
	similarity	<u>29.69 ± 16.43</u>	<u>26.58 ± 18.13</u>	<u>25.60 ± 16.85</u>	<u>19.70 ± 14.29</u>
	time	31.02 ± 28.51	143.34 ± 123.78	27.39 ± 28.05	42.15 ± 56.50

3. Методы поиска источников для энциклопедических статей

Wikipedia

Рассмотренные выше методы позволяют сжимать и объединять информацию из длинных документов. Однако для построения энциклопедической статьи необходимо также найти источники, раскрывающие разные аспекты исходной темы. Если набор источников заранее не задан, задача реферирования становится тесно связана с задачей поиска: качество итогового текста сильно зависит от того, какие документы были найдены.

Обычный одношаговый поиск в такой ситуации часто оказывается недостаточным. Например, пусть требуется собрать материалы о строительстве нового технопарка в Сколково. Один запрос вида “Строительство технопарка Сколково” может найти общие материалы о строительстве (какая компания строит, когда началась стройка и т.д.), но не обязан покрыть все необходимые аспекты: привлечение новых предприятий в Сколково, рост числа рабочих мест, появление новых исследовательских направлений, привлечение инвестиций в Сколково. Эти аспекты связаны с исходной темой, но могут описываться в разных источниках и не содержать одинаковых ключевых слов.

Многошаговый поиск решает эту проблему за счёт последовательного расширения темы. На первом шаге система может найти общие источники, на следующем — выделить из них подтемы, например, про сотрудничающие предприятия, после чего выполнить отдельные запросы по каждой из них. В результате итоговый набор документов оказывается более разнообразным и лучше подходит для построения обзорного энциклопедического текста.

Важно подчеркнуть, что одношаговый поиск не противопоставляется многошаговому, а является его частным случаем. В обозначениях постановки задачи одношаговый поиск получается при $T = 0$, когда последовательность запросов состоит только из исходного запроса q_0 , а множество найденных документов равно $\mathcal{R}_0$. Многошаговый поиск обобщает эту схему: он допускает дополнительные запросы $q_1, \dots, q_T$, которые уточняют или расширяют исходную тему на основе уже найденной информации.

В рамках данной работы многошаговый поиск рассматривается как способ подготовки источников для последующего обзорного реферирования. Далее описываются существующие подходы к multi-hop search и multi-hop RAG, а также предварительные эксперименты, направленные на подготовку русскоязычной поисковой коллекции.

3.1. Обзор существующих подходов

Задача многошагового поиска возникает в тех случаях, когда для ответа на запрос или для раскрытия темы недостаточно одного найденного документа.

Наиболее активно многошаговый поиск исследуется в задачах Multi-hop Question Answering и Retrieval-Augmented Generation. В данной работе рассматриваются несколько подходов, связанных с многошаговым поиском.

Первым таким методом является HopRAG [1]. Он был предложен для решения одной из основных проблем классических RAG-систем: стандартные ретриверы обычно ориентируются на лексическое или семантическое сходство между запросом и документом, но не учитывают логическую связь между документами. В результате система может найти фрагменты, похожие на запрос по словам или эмбедингам, но пропустить документы, которые напрямую не похожи на исходный запрос, однако необходимы для построения цепочки рассуждения. Основная идея HopRAG состоит в представлении корпуса в виде графа. Вершинами графа являются текстовые фрагменты, а рёбра отражают логические переходы между ними. Для построения таких связей используется большая языковая модель: для каждого фрагмента генерируются вопросы или псевдозапросы, которые описывают, к какой информации можно перейти из данного фрагмента или какая информация может вести к нему. Затем фрагменты связываются между собой на основе близости таких псевдозапросов. Таким образом, граф строится не только по поверхностному сходству текстов, но и по потенциальным логическим переходам между ними. У подхода есть существенные ограничения. Во-первых, HopRAG требует заранее зафиксированного корпуса и предварительного построения графа. Каждый документ необходимо обработать с помощью LLM, сгенерировать для него псевдозапросы и построить связи с другими документами. Это делает метод плохо переносимым на сценарий открытого web-поиска, где корпус постоянно меняется и заранее обработать все документы невозможно. Во-вторых, метод в основном оценивается на QA-бенчмарках, где итогом работы является короткий ответ, а не связный обзорный текст. Поэтому прямое применение HopRAG к задаче обзорного реферирования требует адаптации: необходимо не только найти цепочку документов, но и сгруппировать найденные материалы, выделить подтемы и сформировать выводы по ним.

Другой подход к многошаговому поиску предложен в работе TreeHop [2]. Данный метод фокусируется на достижении эффективного обновления запроса в процессе поиска. Многие multi-hop RAG-системы работают по итеративной схеме: система выполняет поиск, передаёт найденные документы в LLM, модель формирует новый запрос, затем этот запрос снова векто-

ризуется и используется для следующего поиска. Такой процесс может быть качественным, но он требует многократных обращений к LLM (как в HopRAG) и поэтому оказывается дорогим и медленным. В TreeHop вместо того, чтобы на каждом шаге заново генерировать текстовый запрос с помощью LLM, обновляются эмбединги запроса на основе уже найденных документов. На первом шаге система получает исходный запрос и извлекает несколько наиболее близких фрагментов. Затем эмбединг запроса модифицируется с учётом найденных документов. При этом авторы стремятся удалить или ослабить ту часть запроса, которая уже была покрыта найденной информацией, и усилить направления, которые могут привести к новым необходимым фактам. Далее обновлённый эмбединг используется для следующего поиска. Так формируется древовидная структура возможных переходов: разные найденные фрагменты могут вести к разным направлениям продолжения поиска. Поскольку метод не требует повторного обращения к LLM для переписывания запроса на каждом шаге, он может существенно снижать задержку и стоимость поиска. Это особенно важно для практических RAG-систем и для больших корпусов, где многошаговый поиск должен выполняться многократно.

В то же время TreeHop также имеет тот же недостаток, что и HopRAG относительно задачи обзорного реферирования. Данный метод ориентирован прежде всего на multi-hop question answering, где цель состоит в нахождении информации для ответа на конкретный вопрос, что не подходит напрямую для создания обзорного реферата для конкретной темы и требует адаптации.

Работа MultiHop-RAG [3] посвящена построению бенчмарка для оценки RAG-систем на многошаговых запросах. Авторы отмечают, что существующие RAG-системы могут показывать приемлемое качество на простых запросах, но испытывают трудности, когда для ответа нужно найти и объединить несколько фрагментов доказательной информации. Для проверки таких сценариев был предложен датасет MultiHop-RAG, основанный на англоязычных новостных статьях.

Ограничение MultiHop-RAG состоит в том, что он ориентирован на англоязычные данные и задачу ответа на вопросы. Для русскоязычного обзорного реферирования готовых бенчмарков существенно меньше, поэтому возникает необходимость либо адаптировать существующие подходы, либо разрабатывать собственную методику оценки.

Отдельным направлением является сжатие найденных документов перед передачей их в языковую модель. В работе BRIEF [4] предлагается подход, при котором длинные документы сжимаются в более компактные тексты, содержащие информацию, важную для многошагового

рассуждения. Это позволяет уменьшить длину контекста, снизить вычислительные затраты и сократить количество нерелевантной информации, отвлекающей модель.

3.2. Предварительные эксперименты с многошаговым поиском

Цель экспериментальной части состояла в предварительной проверке применимости многошагового поиска к задаче подбора обзорного материала. На данном этапе рассматривались отдельные компоненты будущей системы: возможность построения цепочки поиска, поведение существующего multi-hop метода на демонстрационном примере и применимость такого подхода к анализу на примере новостной статьи.

3.3. Демонстрационный запуск TreeHop

В качестве первого эксперимента был рассмотрен демонстрационный запуск метода TreeHop. Данный эксперимент использовался не для оценки качества итоговой системы, а для проверки самой идеи многошагового поиска: система должна не ограничиваться первым найденным документом, а использовать найденную информацию для перехода к следующему шагу. В эксперименте рассматривался вопрос: “Где работает муж Ванессы Трамп?”. Для запуска использовались параметры $N_HOP = 2$ и $TOP_N = 3$. Такой пример хорошо демонстрирует многошаговую природу задачи: для ответа на вопрос недостаточно напрямую найти место работы, сначала необходимо определить, кто является мужем Ванессы Трамп, а затем найти информацию уже об этой сущности. На первом шаге поиска система извлекает документы, связанные с исходным вопросом. Далее найденные фрагменты используются для обновления направления поиска. В результате на одном из следующих шагов система переходит к сущности “Дональд Трамп младший”, что позволяет приблизиться к ответу на исходный вопрос. Получившийся граф продемонстрирован на рисунке 1. Однако стоит отметить, что прямое применение TreeHop к обзорному поиску требует дополнительной адаптации.

3.4. Качественный анализ новостного примера

Следующим этапом была попытка применить идею многошагового поиска напрямую, используя новостной поисковик. В качестве исходного запроса рассматривалась новость о том, что Туркмения готова увеличить поставки природного газа. Один из возможных ожидаемых результатов представлен на рисунке 2.

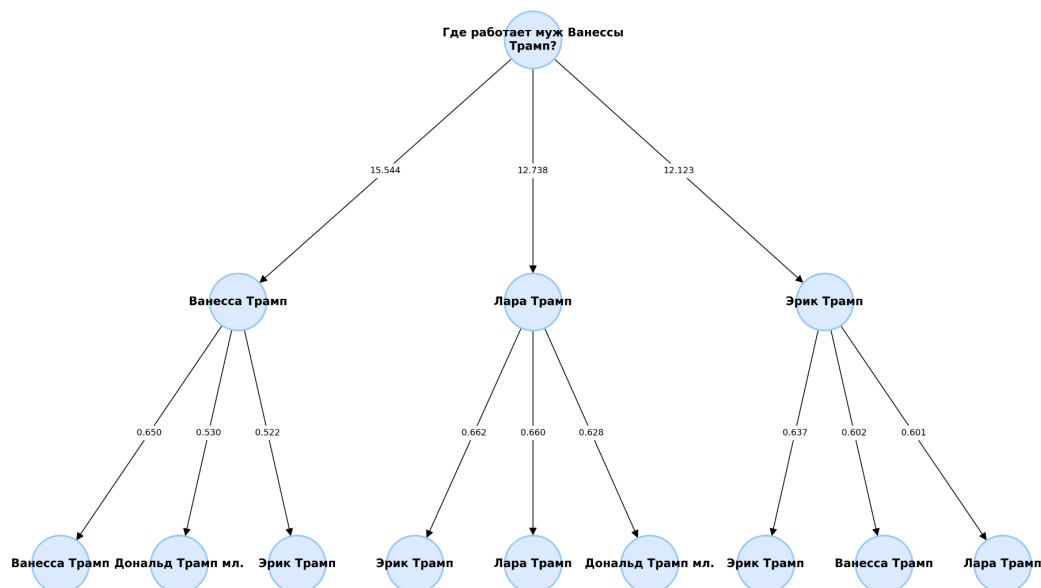
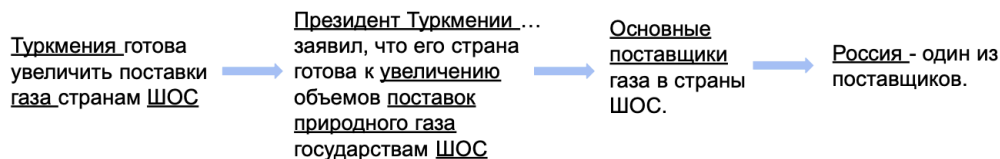


Рис. 1: Пример многошагового поиска в демонстрационном запуске TreeNop



Финальный вывод:

Туркмения собирается потеснить РФ на азиатском рынке природного газа

Рис. 2: Пример цепочки запросов

В ходе тестового запуска была предпринята попытка построить цепочку запросов с использованием новостного поисковика. Однако результат показал ограниченность прямого применения такого подхода. В место последовательного расширения темы модель стала уходить в слишком конкретные уточнения. Например, дальнейшие вопросы смещались к частным деталям, таким как отдельные газопроводы и инфраструктурные маршруты. В результате система не смогла перейти к формированию обзорного реферата и не построила достаточно широкую цепочку тематических связей.

Данный запуск показывает необходимость создания более совершенных методов построения обзорного реферата, поскольку при прямых запросах LLM предпочитают уходить в

конкретику, что, в свою очередь, не дает необходимой информации для построения всеобъемлющего обзорного реферата.

3.5. Подготовка русскоязычной поисковой коллекции

Для проведения экспериментов с многошаговым поиском необходима собственная поисковая среда, позволяющая извлекать документы по автоматически построенным запросам. Использование готовых QA-бенчмарков не полностью соответствует рассматриваемой задаче, поскольку в них, как правило, уже задан ограниченный набор документов и требуется получить короткий ответ на вопрос. К тому же, в основном данные бенчмарки сфокусированы на английском языке.

Поэтому для дальнейших экспериментов требуется сформировать русскоязычную текстовую коллекцию, приближенную к условиям web-поиска. Такая коллекция должна содержать большое количество документов из разных источников и тематических областей. Это позволит проверять не только способность модели генерировать подзапросы, но и качество самого поиска: какие документы находятся по каждому шагу, насколько они связаны с исходной темой и помогают ли они построить обзорный вывод.

В качестве основы для такой коллекции в работе рассматривается корпус FineWeb2¹. FineWeb2 представляет собой крупный многоязычный корпус веб-текстов, построенный на основе данных CommonCrawl. Для задачи данной работы важна русскоязычная часть корпуса, поскольку целевой сценарий предполагает поиск обзорных материалов по русскоязычным новостям и веб-документам.

Следует отметить, что FineWeb2 сам по себе не является датасетом для многошагового поиска или обзорного реферирования. Он не содержит готовых цепочек запросов, разметки релевантности или эталонных обзорных рефератов. В рамках данной работы он используется как источник документов, на основе которого может быть построен поиск.

Информация об исходном датасете представлена в таблице 3.

Поскольку данный датасет имеет огромный размер по памяти, было принято решение скачать его сжатую версию. Для удобной работы с датасетом, все данные записывались в jsonl файлы по 10000 записей на файл. Данная версия была получена путем ограничения количества текстов до 10 штук на каждый домен. Статистика полученного таким образом датасета представлена в таблице 4.

¹<https://huggingface.co/collections/HuggingFaceFW/fineweb2>

Подмножество	Слова	Документы	Размер на диске
rus_Cyrl	588,579,493,780	699,083,579	1.81 TB
FineWeb2 total	3,339,271,691,958	5,018,505,566	8.58 TB

Таблица 3: Статистика корпуса FineWeb2 и его русскоязычного подмножества

Показатель	Значение
Число документов	35.6 млн
Уникальных сайтов	7.12 млн
Размер коллекции на диске	233 ГБ
Средняя длина документа	3582 символа
Временной диапазон	2013–2024
Документы длиной 200–999 символов	28.8%
Документы длиной 1k–5k символов	54.7%
Документы длиной 5k–20k символов	14.5%
Документы длиной >20k символов	2.0%

Таблица 4: FineWeb2 подмножество сайтов

3.6. Автоматическая оценка качества источников

Поскольку FineWeb2 представляет собой большой корпус веб-текстов, его непосредственное использование для поисковых экспериментов связано с рядом проблем. В корпусе могут встречаться документы различного качества: новостные статьи, образовательные материалы, коммерческие страницы, автоматически сгенерированные тексты, рекламные описания, фрагменты форумов и другие типы веб-контента. Для задачи подбора обзорного материала особенно важно, чтобы поисковая система отдавала предпочтение источникам, содержащим связную, информативную и достаточно надёжную информацию. В связи с этим был проведён предварительный эксперимент по автоматической оценке качества сайтов. Из загруженной части коллекции было случайным образом выбрано около 10 тысяч примеров.

Для разметки использовалась модель Qwen3-235B. Модель получала на вход набор фрагментов и должна была выставить оценки по нескольким критериям по шкале от 0 до 5. Каждый критерий отражал отдельное свойство источника: образовательную ценность, качество языка, коммерческую направленность, информационную надёжность и потенциальные риски содер-

жания. Полученные оценки в дальнейшем могут использоваться для обучения более быстрого метода классификации, например, на основе моделей семейства BERT.

3.7. Статистика автоматической оценки качества сайтов

Полученные с помощью LLM оценки использовались как псевдоразметка для дальнейшего обучения более лёгкой модели семейства BERT. Такой подход позволяет один раз выполнить дорогую оценку небольшой выборки с помощью большой языковой модели, а затем обучить компактный классификатор, способный быстрее оценивать качество сайтов на существенно большем объёме данных.

В итоговую размеченную выборку вошло 14611 сайтов. Для каждого сайта были получены оценки по шести критериям: коммерческая направленность, образовательная ценность, риск вредного содержания, информационная надёжность, качество языка и эвристическая оценка нейтральности текста. Распределение оценок на уровне сайтов приведено в таблице 5.

Критерий	Сред.	Ст. откл.	0	1	2	3	4	5
commercial_intensity	2.354	1.760	3391	1888	2433	1904	3042	1953
educational_value	1.497	1.281	4230	3363	3919	1807	1208	84
harmful_content_risk	0.374	0.968	12076	1080	614	316	418	107
information_reliability	2.416	1.098	1065	1421	4756	5255	1961	153
language_quality	3.060	1.428	937	1526	2342	2668	5201	1937
rf_political_compliance	4.423	1.271	686	236	424	477	1828	10960

Таблица 5: Распределение LLM-оценок качества сайтов

Такая статистика показывает, что загруженная часть FineWeb2 неоднородна по качеству. Поэтому использование дополнительного классификатора качества источников является важным этапом подготовки поисковой коллекции: он может применяться для фильтрации низкокачественных сайтов или для учёта качества источника при ранжировании результатов поиска.

3.8. Применение модели BERT для классификации сайтов

Автоматическая оценка сайтов с помощью большой языковой модели позволяет получить достаточно подробную разметку, однако такой подход плохо масштабируется на весь объём

ём коллекции. Каждый вызов LLM требует значительных вычислительных ресурсов, поэтому прямое применение такой модели ко всем сайтам FineWeb2 является неэффективным. В связи с этим в работе был использован двухэтапный подход: сначала небольшая выборка сайтов размечается большой языковой моделью, после чего на полученных оценках обучается более лёгкая модель-классификатор.

В качестве такой модели использовалась `deepvk/RuModernBERT-base`². Для каждого из критериев обучалась отдельная модель, предсказывающая оценку от 0 до 5.

Входом модели являлся текст, составленный из нескольких фрагментов одного сайта. Для каждого сайта использовалось не более шести текстовых фрагментов, причём длина каждого фрагмента ограничивалась 4000 символами. Таким образом, модель получала несколько примеров содержимого сайта, что позволяло приблизительно оценивать источник в целом. В обучающей выборке для каждого критерия использовалось до 10000 сайтов. Разбиение на обучающую и валидационную части выполнялось в соотношении 80/20.

Формально задача была сведена к регрессии на шкале от 0 до 5. Метки нормировались к диапазону $[0, 1]$, после чего для обучения использовалась функция потерь `BCEWithLogitsLoss`. На этапе предсказания выход модели преобразовывался с помощью сигмоиды, умножался на 5 и округлялся до ближайшего целого значения. Таким образом, модель могла использоваться как для получения непрерывной оценки качества, так и для классификации по исходной шестибалльной шкале.

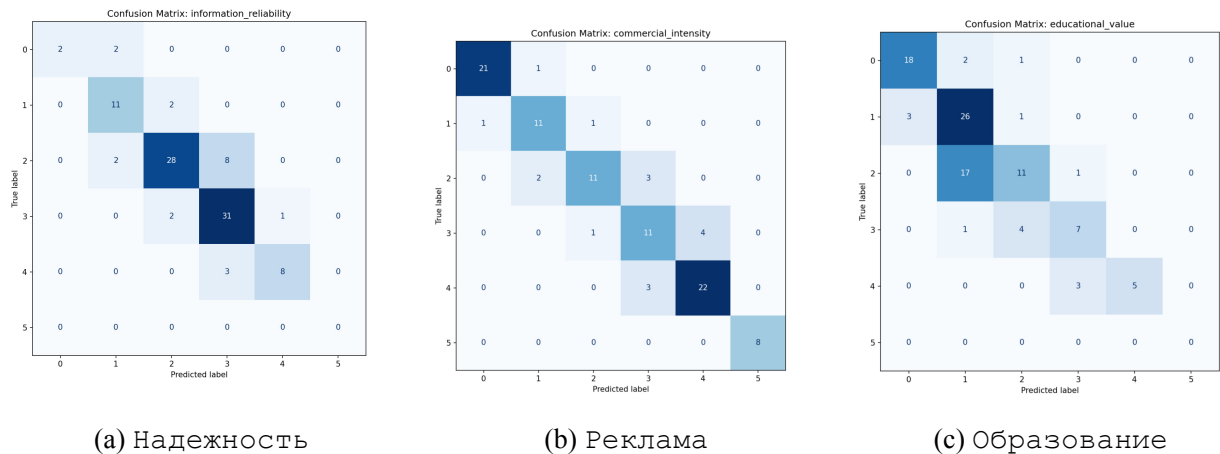


Рис. 3: Матрицы ошибок модели RuModernBERT

Для качественного анализа были построены матрицы ошибок на 1000 случайных при-

²<https://huggingface.co/deepvk/RuModernBERT-base>

мерах. В качестве целевых значений использовались оценки, полученные от LLM. На рисунках 3а–3с показаны результаты для трёх критериев, которые оказались наиболее показательными: информационная надёжность, коммерческая направленность и образовательная ценность.

По критерию `commercial_intensity` модель показывает наиболее устойчивое поведение. Большая часть предсказаний сосредоточена на диагонали матрицы ошибок, то есть модель часто воспроизводит оценку LLM точно. Кроме того, большинство ошибок приходится на соседние классы. Это означает, что модель редко путает, например, полностью некоммерческий сайт с сайтом с очень высокой коммерческой направленностью.

Для критерия `information_reliability` также наблюдается концентрация значений около диагонали. Модель достаточно хорошо различает средние значения шкалы, особенно классы 2 и 3. При этом крайние классы представлены слабее: например, оценка 5 практически отсутствует в рассматриваемой выборке. Это связано не только с качеством модели, но и с распределением исходной LLM-разметки: для веб-корпуса общего назначения очень высокая информационная надёжность встречается редко. Поэтому модель в первую очередь обучается различать низкую, среднюю и умеренно высокую надёжность.

Наиболее сложным оказался критерий `educational_value`. Матрица ошибок показывает, что модель часто занижает оценки для сайтов со средней образовательной ценностью. Например, часть объектов с истинной оценкой 2 предсказывается как 1. Это может объясняться как субъективностью самого критерия, так и неоднородностью сайтов: отдельные фрагменты могут содержать полезную информацию, но не образовывать системный образовательный материал. Кроме того, в исходной разметке распределение по этому критерию смещено в сторону низких оценок, что затрудняет обучение для высоких классов. В целом полученные результаты показывают, что модель семейства BERT способна приближённо воспроизводить оценки большой языковой модели. При этом качество зависит от критерия. Более явно выраженные свойства, такие как коммерческая направленность, оказываются проще для обучения. Более субъективные признаки, такие как образовательная ценность, предсказываются хуже и требуют либо большего объёма разметки, либо объединения соседних классов.

3.9. Использование тематических тегов сайтов

Последним этапом практической части стала попытка улучшить классификацию сайтов за счёт использования тематических тегов. Предыдущие эксперименты показали, что модель, обученная только на текстовых фрагментах сайта, способна приближённо воспроизводить LLM-

оценки, однако для некоторых критериев качество остаётся ограниченным. В частности, образовательная ценность является достаточно субъективным признаком, который может зависеть не только от качества отдельных фрагментов, но и от общей тематики сайта. Для решения этой проблемы была начата работа по извлечению и агрегации тематических тегов. Предположение состоит в том, что тематика сайта может быть полезным дополнительным признаком для оценки его качества. Небольшая статистика по полученным тегам представлена в таблице 6.

Для уменьшения числа разрозненных тематических меток применяется агломеративная кластеризация. Перед построением эмбедингов теги нормализуются: удаляется символ #, подчеркивания заменяются пробелами, а текст приводится к нижнему регистру. Далее для каждого тега формируется фраза вида “Сайт посвящен тегу: ...”, по которой строится векторное представление.

Цель такого подхода состоит в том, чтобы сгруппировать близкие по смыслу теги и использовать их как дополнительные признаки при обучении классификатора.

Тег	Число вхождений
#строительные_материалы	3328
#дизайн_интерьера	2400
#образование	2312
#игровые_автоматы	2198
#технические_характеристики	1598
#строительство	1580
#туризм	1505
#уход_за_кожей	1360
#недвижимость	1328
#интернет_магазин	1252
#здоровье	1168
#промышленное_оборудование	1121
#кулинария	1047

Таблица 6: Пример некоторых наиболее частотных тегов

Таким образом, практическая часть работы включала подготовку русскоязычной подколлекции FineWeb2, получение LLM-разметки качества сайтов, обучение более лёгкой модели для

воспроизведения этой разметки и начальную работу по добавлению тематических признаков.

4. Заключение

В работе была рассмотрена задача многошагового поиска обзорного материала по исходной теме, новости или факту. В отличие от обычного поиска, здесь требуется не просто найти документы по одному запросу, а построить цепочку связанных подзапросов, которая помогает раскрыть разные аспекты темы и сформировать итоговый обзор.

Были рассмотрены современные подходы к multi-hop search и multi-hop RAG, в том числе HopRAG и TreeHop. Эти методы показывают, что многошаговый поиск может строиться либо через явные связи между документами, либо через последовательное обновление запроса. При этом большинство существующих работ ориентировано на задачу ответа на вопросы, а не на обзорное реферирование.

Практическая часть работы была направлена на подготовку основы для дальнейших экспериментов. Была загружена часть русскоязычной коллекции FineWeb2, проведена LLM-разметка качества сайтов, обучена модель RuModernBERT для более быстрой оценки источников, а также начата работа с тематическими тегами. Эти этапы нужны для того, чтобы в дальнейшем строить поисковую систему не по произвольной шумной коллекции, а по предварительно отфильтрованным и размеченным данным.

В дальнейшем планируется разработать методику оценки multi-hop подходов для задачи обзорного реферирования, адаптировать существующие методы к условиям web-поиска и протестировать их на крупной русскоязычной коллекции FineWeb2.

Список литературы

- [1] *Liu, Hao*. HopRAG: Multi-Hop Reasoning for Logic-Aware Retrieval-Augmented Generation / Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, Wentao Zhang. — arXiv:2502.12442. — 2025. <https://arxiv.org/abs/2502.12442>.
- [2] *Li, Zhonghao*. TreeHop: Generate and Filter Next Query Embeddings Efficiently for Multi-hop Question Answering / Zhonghao Li, Kunpeng Zhang, Jinghui Ou, Shuliang Liu, Xuming Hu. — arXiv:2504.20114. — 2025. <https://arxiv.org/abs/2504.20114>.
- [3] *Tang, Yixuan*. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries / Yixuan Tang, Yi Yang. — arXiv:2401.15391. — 2024. <https://arxiv.org/abs/2401.15391>.
- [4] *Li, Yuankai*. BRIEF: Bridging Retrieval and Inference for Multi-hop Reasoning via Compression / Yuankai Li, Jia-Chen Gu, Di Wu, Kai-Wei Chang, Nanyun Peng. — arXiv:2410.15277. — 2024. <https://arxiv.org/abs/2410.15277>.